

第 11 讲: 梯度下降

为什么沿负梯度方向可以让损失下降

@ Frano

● FranSpace

本讲摘要 / Why it matters

梯度下降 (Gradient Descent, GD) 是最基础、最重要的优化思想之一。它的核心思想很简单: 如果梯度表示损失函数上升最快的方向, 那么负梯度方向就是局部下降最快的方向。神经网络训练中, 反向传播负责计算梯度, 梯度下降则提供一种使用梯度更新参数的基本规则。理解梯度下降, 是理解 SGD、Momentum、Adam、L-BFGS 以及 PINN 训练策略的基础。

11.1 从“下山”理解梯度下降

如果把损失函数想象成一座山, 那么训练神经网络就是从山上的某个位置出发, 逐步向山谷走去。山上的高度对应损失函数值, 位置对应模型参数。我们希望找到一个较低的位置, 也就是让损失函数尽量小。

假设当前位置是 θ_k , 损失函数为 $L(\theta)$ 。如果我们知道当前位置附近哪个方向上升最快, 那么反方向通常就是下降最快的方向。这个“上升最快的方向”就是梯度:

$$\nabla_{\theta} L(\theta_k).$$

因此, 梯度下降的更新思想就是:

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

其中 η 是学习率, 控制每次沿负梯度方向走多远。

分析 / Insight

梯度下降并不是一次找到最优解, 而是不断重复“小步调整”。每一步只利用当前位置附近的局部信息, 因此它是一种迭代优化方法。

11.2 一维函数中的梯度下降

为了更直观地理解梯度下降，先考虑一个只有一个参数的函数 $L(\theta)$ 。此时梯度就是普通导数：

$$\frac{dL}{d\theta}.$$

若

$$\frac{dL}{d\theta} > 0,$$

说明随着 θ 增大，损失函数在增大。为了让损失下降，应该减小 θ 。若

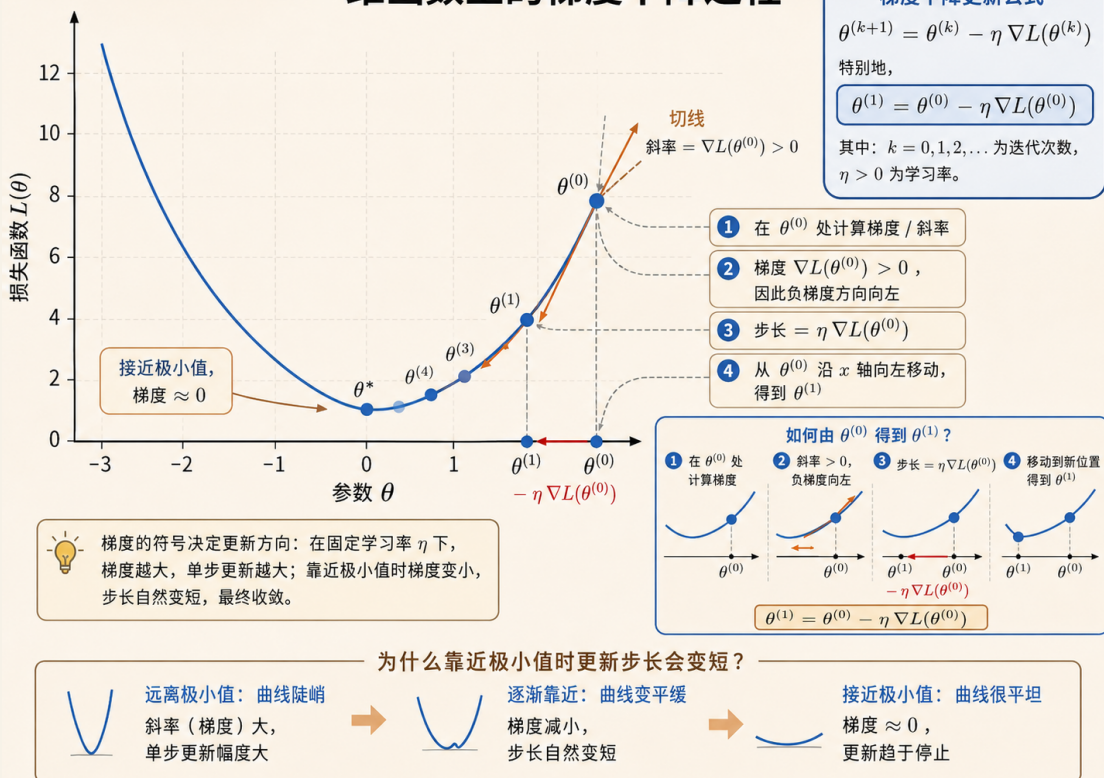
$$\frac{dL}{d\theta} < 0,$$

说明随着 θ 增大，损失函数在减小。为了让损失下降，应该增大 θ 。

这两种情况都可以统一写成：

$$\theta_{k+1} = \theta_k - \eta \frac{dL}{d\theta}.$$

一维函数上的梯度下降过程



图中每个点表示一次迭代后的参数位置。箭头表示参数更新方向。可以看到，参数并不是一次跳到最小点，而是根据当前梯度逐步向低损失区域移动。

11.3 二维参数中的梯度方向

如果参数不止一个，例如

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix},$$

那么损失函数是多元函数

$$L = L(\theta_1, \theta_2).$$

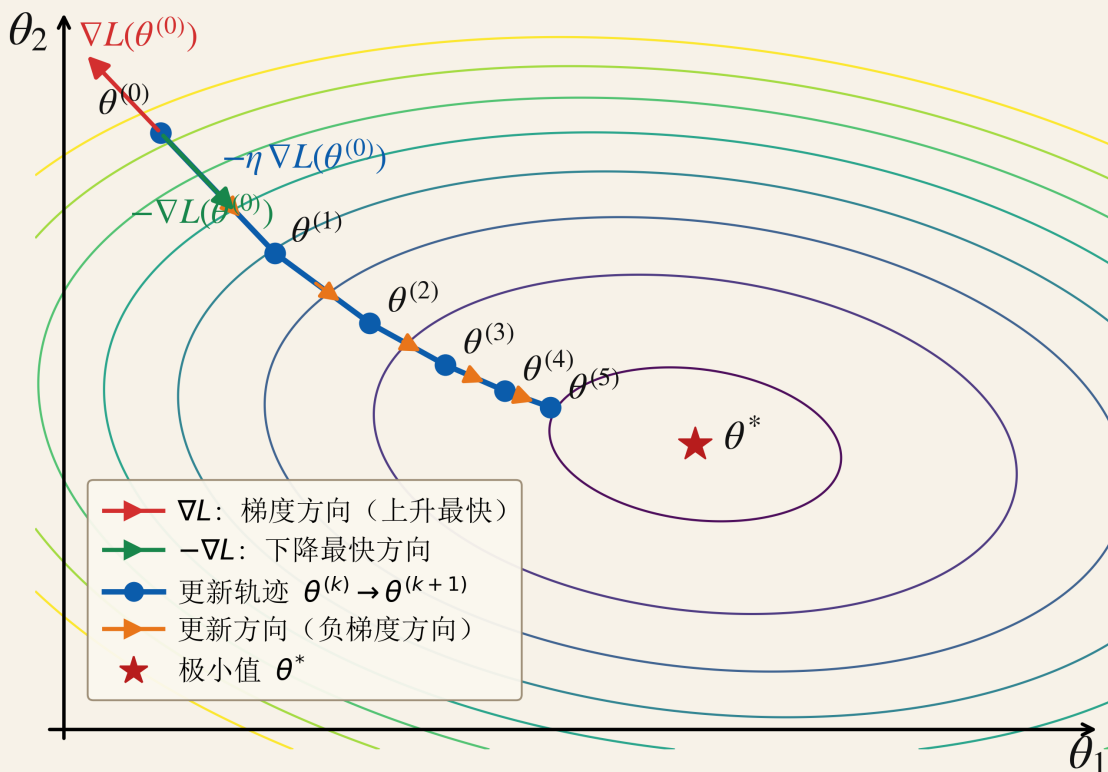
此时梯度是一个向量：

$$\nabla_{\theta} L = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \end{bmatrix}.$$

梯度向量指向损失函数局部上升最快的方向，因此负梯度方向

$$-\nabla_{\theta} L$$

就是局部下降最快的方向。二维情况下，可以用等高线来理解这个过程。等高线表示损失函数取相同值的位置，梯度方向与等高线垂直。



备注 / Note

梯度下降中的“最快下降”只是在当前位置附近的局部意义上成立。它并不保证一步到达全局最优，也不保证一定不会遇到局部极小值、鞍点或平坦区域。

11.4 梯度下降的更新公式

梯度下降的标准更新公式为

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

这个公式中包含三个关键部分：

- θ_k ：第 k 次迭代时的参数；
- $\nabla_{\theta} L(\theta_k)$ ：当前参数处损失函数的梯度；
- η ：学习率，决定每次更新的步长。

如果把公式拆开理解，可以写成：

$$\text{新参数} = \text{旧参数} - \text{学习率} \times \text{梯度}.$$

其中，负号非常重要。梯度方向是损失函数上升最快方向，而我们希望损失下降，所以要沿梯度的反方向更新。

Example / 例子

假设某个参数当前为 $w = 2.0$ ，损失对它的梯度为

$$\frac{\partial L}{\partial w} = 3.0.$$

若学习率为 $\eta = 0.1$ ，则梯度下降更新为

$$w_{\text{new}} = 2.0 - 0.1 \times 3.0 = 1.7.$$

因为梯度为正，说明增大 w 会使损失增大，所以更新后 w 被减小。

11.5 学习率的作用

学习率 (Learning Rate, LR) 是梯度下降中最重要的超参数之一。它决定每次参数更新的步长。

如果学习率太小，参数每次只移动一点点，训练会非常慢；如果学习率太大，参数可能直接跨过低损失区域，导致震荡甚至发散。

分析 / Insight

学习率不是越大越好，也不是越小越安全。合适的学习率应该在训练速度和稳定性之间取得平衡。

11.6 梯度下降和反向传播的关系

前一讲讲过，反向传播负责计算损失函数对参数的梯度：

$$\nabla_{\theta} L.$$

而梯度下降负责使用这个梯度更新参数：

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L.$$

因此二者的关系可以总结为：

反向传播算梯度，梯度下降用梯度更新参数。

在 PyTorch 中，通常对应为：

- `loss.backward()`：反向传播，计算梯度；
- `optimizer.step()`：优化器根据梯度更新参数；
- `optimizer.zero_grad()`：清空上一轮梯度。

需要注意的是，严格来说，现代代码中使用的优化器不一定是最朴素的梯度下降。例如 Adam、RMSProp、L-BFGS 都有更复杂的更新规则。但它们都依赖梯度信息，梯度下降是理解这些优化器的基础。

11.7 批量梯度下降、随机梯度下降和小批量梯度下降

前面讲梯度下降时，我们默认写的是

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

但这里有一个容易被忽略的问题：这个损失函数 $L(\theta)$ 到底是用多少数据算出来的？

假设训练集中一共有 N 个样本：

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}.$$

每个样本都可以产生一个单独的损失：

$$L_i(\theta) = L(f_\theta(\mathbf{x}_i), \mathbf{y}_i).$$

如果把所有样本的损失平均起来，就得到整个训练集上的总损失：

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta).$$

因此，总梯度可以写成

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_\theta L_i(\theta).$$

关键区别就在这里：每次更新参数时，我们到底使用全部 N 个样本来计算梯度，还是只用其中一部分样本来估计梯度？根据这个选择，就得到批量梯度下降、随机梯度下降和小批量梯度下降三种做法。

• 11.7.1 批量梯度下降

批量梯度下降 (Batch Gradient Descent) 每次更新参数时，都使用整个训练集来计算损失和梯度。

也就是说，每一步都使用

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta),$$

并计算

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_\theta L_i(\theta).$$

然后更新参数：

$$\theta_{k+1} = \theta_k - \eta \frac{1}{N} \sum_{i=1}^N \nabla_\theta L_i(\theta_k).$$

它的直观理解是：每次更新前，先问一遍整个训练集：“当前参数整体表现怎么样？”然后根据所有样本给出的平均意见来更新参数。

Example / 例子

假设训练集中有 10000 个样本，批量梯度下降每更新一次参数，都要把这 10000 个样本全部送进网络，计算所有样本的 loss，再求平均梯度，最后才更新一次参数。

备注 / Note

批量梯度下降的特点是“方向稳，但更新慢”。它每一步都很认真地看完整个训练集，但也正因为如此，每一步计算成本很高。

• 11.7.2 随机梯度下降

随机梯度下降 (Stochastic Gradient Descent, SGD) 的想法刚好相反：每次更新参数时，只随机选取一个样本来计算梯度。

假设这一步随机选中了第 i 个样本，那么只计算

$$L_i(\theta)$$

以及它的梯度

$$\nabla_{\theta} L_i(\theta).$$

参数更新为

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L_i(\theta_k).$$

它的直观理解是：每次更新前，不再询问整个训练集，而是随机找一个样本问：“你觉得当前参数该怎么改？”然后就根据这个样本的意见更新参数。

Example / 例子

假设训练集中有 10000 个样本，随机梯度下降每次只抽取其中 1 个样本，计算这个样本的 loss 和梯度，然后马上更新一次参数。

随机梯度下降的优点是每次更新非常快，因为只需要计算一个样本。它更新频率高，参数可以很快开始移动。

但是它的问题是梯度噪声很大。单个样本给出的梯度不一定能代表整个训练集的平均方向。这个样本可能建议参数往左走，另一个样本可能建议往右走。因此，SGD 的训练路径通常会比较抖动，loss 曲线也不如批量梯度下降平滑。

不过，这种噪声并不全是坏事。一定程度的随机性有时可以帮助模型跳出一些较差的局部区域，也可能提升泛化能力。但如果噪声太大，训练就会不稳定。

分析 / Insight

随机梯度下降的特点是“更新快，但方向 noisy”。它每次只看一个样本，所以计算便宜，但更新方向不够稳定。

• 11.7.3 小批量梯度下降

小批量梯度下降 (Mini-Batch Gradient Descent) 是最常用的折中方法。它每次不使用全部样本，也不只使用一个样本，而是使用一小批样本。

设当前 mini-batch 为

$$\mathcal{B} = \{i_1, i_2, \dots, i_B\},$$

其中 B 是 batch size。那么 mini-batch 损失为

$$L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} L_i(\theta).$$

对应梯度为

$$\nabla_{\theta} L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} L_i(\theta).$$

参数更新为

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L_{\mathcal{B}}(\theta_k).$$

它的直观理解是：每次更新前，不问整个训练集，也不只问一个样本，而是随机抽一小组样本，听取这组样本的平均意见，再更新参数。

Example / 例子

假设训练集中有 10000 个样本，batch size 取 100。那么每次更新参数时，只使用其中 100 个样本计算平均 loss 和平均梯度。一个 epoch 中大约会有

$$\frac{10000}{100} = 100$$

次参数更新。

小批量梯度下降的优点是兼顾了计算效率和梯度稳定性。相比批量梯度下降，它每次只计算一部分样本，更新更频繁；相比随机梯度下降，它使用多个样本的平均梯度，方向更稳定。

这也是现代深度学习中最常用的训练方式。很多时候我们口头上说“使用 SGD 训练”，实际代码中往往使用的是 mini-batch SGD，而不是每次只用一个样本的严格 SGD。

• 11.7.4 三者的对比

三种方法的区别可以总结如下：

表 1: 批量梯度下降、随机梯度下降和小批量梯度下降对比

方法	每次使用的数据量	梯度特点	主要特点
批量梯度下降	全部训练集	最稳定, 噪声小	每次更新成本高, loss 曲线平滑, 适合小数据或 full-batch 场景
随机梯度下降	单个样本	噪声最大	更新快但抖动大, 严格形式现在较少直接使用
小批量梯度下降	一小批样本	稳定性适中	最常用, 兼顾计算效率和梯度稳定性

从计算成本看:

批量梯度下降 > 小批量梯度下降 > 随机梯度下降

是指每一次参数更新的计算成本。

从梯度稳定性看:

批量梯度下降 > 小批量梯度下降 > 随机梯度下降。

从更新频率看, 如果以一个 epoch 为单位, 批量梯度下降只更新一次; 小批量梯度下降更新多次; 随机梯度下降更新 N 次。

● 11.7.5 batch size 对训练的影响

小批量梯度下降中的 batch size 很重要。它决定每次用多少样本估计梯度。

如果 batch size 较小:

- 每次计算快;
- 更新频率高;
- 梯度噪声较大;
- loss 曲线可能更抖;
- 有时泛化能力较好。

如果 batch size 较大:

- 梯度估计更稳定;
- 计算更适合并行;
- 每次更新成本更高;

- 参数更新次数减少；
- 可能需要调整学习率。

这说明 batch size 不是越大越好，也不是越小越好。它需要结合显存、训练稳定性、学习率和任务规模一起考虑。

11.8 梯度下降可能遇到的问题

梯度下降虽然思想简单，但在复杂神经网络训练中会遇到很多问题。

• 11.8.1 局部极小值

损失函数可能有多个低谷。梯度下降从不同初始点出发，可能收敛到不同位置。对非凸问题来说，很难保证一定找到全局最优。

• 11.8.2 鞍点

鞍点是某些方向像谷底、某些方向像山顶的位置。在高维神经网络中，鞍点可能比局部极小值更常见。梯度在鞍点附近可能很小，导致训练变慢。

• 11.8.3 平坦区域

如果损失函数在某些区域变化很慢，梯度很小，参数更新也会很慢。这会导致训练停滞。

• 11.8.4 陡峭峡谷

有些损失面在一个方向很陡，在另一个方向很平缓。普通梯度下降可能在陡峭方向来回震荡，而在真正通向最优点的方向前进缓慢。

这些问题也是后续引入 Momentum、Adam、L-BFGS 等优化器的重要原因。

11.9 梯度下降在 PINN 中的作用

在 PINN 中，梯度下降的对象仍然是神经网络参数 θ ，但损失函数更加复杂。以固体力学 PINN 为例，总损失可能写成

$$L(\theta) = \lambda_r L_r + \lambda_D L_D + \lambda_N L_N + \lambda_{\text{data}} L_{\text{data}}.$$

其中，PDE 残差损失 L_r 可能依赖网络输出对坐标的一阶、二阶甚至更高阶导数。这意味着 PINN 的梯度不仅来自数据误差，还来自物理方程和边界条件。

在 PINN 中，梯度下降面临几个特殊困难：

- 不同 loss 项的数量级可能差别很大；

- PDE 残差和边界损失可能竞争；
- 高阶自动微分会使计算图更复杂；
- 训练点分布会影响梯度方向；
- 总 loss 下降不代表每个物理约束都满足。

因此，在 PINN 中使用梯度下降或 Adam 等一阶优化器时，必须关注各个子 loss 的下降情况，而不能只看总 loss。

11.10 为什么普通梯度下降不够

朴素梯度下降只利用当前梯度方向。它不知道过去几步走过哪里，也不知道当前损失面是否陡峭或弯曲。因此，在复杂问题中它可能效率较低。

例如：

- 如果梯度方向频繁变化，普通梯度下降容易震荡；
- 如果不同参数方向尺度差别很大，更新会不平衡；
- 如果学习率设置不合适，训练可能很慢或发散；
- 如果损失函数条件数很差，收敛会非常缓慢。

这就是为什么后续需要学习 Momentum、RMSProp、Adam 和 L-BFGS。它们可以看作在梯度下降基础上的改进：

- Momentum 利用历史梯度方向；
- RMSProp 和 Adam 根据梯度大小自适应调整步长；
- L-BFGS 近似利用曲率信息。

11.11 常见误区

关于梯度下降，初学者常见误区包括：

- 误区一：梯度下降每一步都一定让 loss 下降。

如果学习率过大，loss 可能上升甚至发散。

- 误区二：梯度方向是下降最快方向。

梯度方向是上升最快方向，负梯度方向才是局部下降最快方向。

- 误区三：学习率越小越好。

学习率太小会导致训练非常慢，甚至看起来像不收敛。

- 误区四：总 loss 下降就说明 PINN 训练成功。

对 PINN 来说，总 loss 下降还不够，还要看 PDE loss、边界 loss、位移误差和应力误差。

- 误区五：优化器能弥补所有问题。

如果控制方程、边界条件、本构关系或自动微分写错，再好的优化器也无法得到正确物理解。

11.12 实践建议

在实际训练神经网络或 PINN 时，可以遵循以下建议：

- 先用较简单的网络和较小问题验证 loss 是否能正常下降；
- 分别打印各个子 loss，不要只看总 loss；
- 对学习率进行小范围试验，观察 loss 是否平稳下降；
- 如果 loss 剧烈震荡，优先尝试减小学习率；
- 如果 loss 下降非常慢，可以尝试适当增大学习率或使用 Adam；
- 对 PINN，检查梯度是否真实来自 PDE 残差，而不是计算图被截断；
- 在切换 L-BFGS 前，通常可以先用 Adam 预训练到较合理区域。

11.13 一页总结

- 梯度下降 (Gradient Descent, GD) 是一种利用梯度信息最小化损失函数的基本优化方法。
- 梯度 $\nabla_{\theta} L$ 表示损失函数局部上升最快方向，负梯度方向表示局部下降最快方向。
- 梯度下降更新公式为

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

- 学习率 (Learning Rate, LR) 控制每次参数更新步长，过小会慢，过大可能震荡或发散。
- 反向传播负责计算梯度，梯度下降或优化器负责使用梯度更新参数。
- 根据每次使用的数据量，可以分为批量梯度下降、随机梯度下降和小批量梯度下降。
- 在 PINN 中，梯度来自 PDE 残差、边界条件、数据项等多个 loss 项，因此必须分别监控子 loss。
- 普通梯度下降是理解 Adam、Momentum、RMSProp 和 L-BFGS 的基础，但在复杂 PINN 问题中通常需要更强的优化策略。